

UNIVERSITY OF COMPUTER STUDIES, YANGON
2025-2026 ACADEMIC YEAR



CHESS PUZZLES DIFFICULTY CLASSIFICATION

Semester IX

DATA AND KNOWLEDGE MINING (IS-212) PROJECT

Submitted By

YKPT-22466

Mg Hein Htet Zaw

September, 2026

B.C.Sc.(SE)

ACKNOWLEDGEMENT

I would like to thank my course instructor, Dr. Hsu Myat Mo, for guiding the class through the IS-212 lectures and for answering my questions during lab sessions. The lectures on classification, association rule mining, and model evaluation formed the foundation of this project. The feedback I received during the planning phase helped me avoid several potential mistakes.

Table of Contents

ACKNOWLEDGEMENT.....	1
CHAPTER - 1 INTRODUCTION	2
1.1 Project Background.....	4
1.2 Objectives of the Project.....	4
1.3 Tools and Environment.....	5
CHAPTER - 2 DATASET DESCRIPTION AND EXPLORATORY DATA ANALYSIS.....	4
2.1 Dataset Overview.....	6
2.2 Data Quality Check.....	6
2.3 Target Variable and Difficulty Classes.....	7
2.4 Relationships Between Features	8
2.5 Potentially Leaking Themes	11
CHAPTER - 3 DATA PREPROCESSING AND DESCRIPTIVE MINING.....	7
3.1 The Removal of Columns and the Process of Feature Engineering.....	12
3.2 Encoding, Scaling and the Final Feature Matrix	13
3.3 The Use of the Apriori Algorithm for Association Rule Mining.....	13
3.4 Rules Indicating Difficulty Classes	14
CHAPTER - 4 PREDICTIVE MINING AND EVALUATION.....	10
4.1 Models and Hyperparameter Search.....	16
4.2 A Problem That Was Resolved While Mining	16
4.3 Cross-Validation Results	17
4.4 Test Set Results.....	18
4.5 Feature Importance	20
4.6 The Leakage Audit: Eliminating the Themes Related to the Solution Length	21
4.7 Error Analysis	22

4.8 The Implemented System	23
CHAPTER - 5 CONCLUSION.....	15
5.1 The Benefits of this Project.....	24
5.2 The Disadvantages of This Project / Limitations	25
REFERENCES.....	26

CHAPTER - 1

INTRODUCTION

This report presents my individual project for the IS-212 Data and Knowledge Mining course. The project applies both descriptive and predictive data mining techniques to a real dataset, implemented in a Jupyter notebook on Google Colab. All results and figures in the following chapters were generated directly from the raw CSV file using this notebook.

1.1 Project Background

Lichess is a free and open-source chess server, and in addition to hosting games it keeps a large public database of chess puzzles. A puzzle consists of a single position taken from a real game in which one side has the possibility of gaining material or delivering checkmate, and each puzzle on the site is given a Glicko rating which indicates the level of strength that a player needs to solve it. The database also records metadata including how many times the puzzle has been played, what tactical themes occur in the solution sequence, and the FEN string which describes the board.

The rating value cannot be used as an input feature, as it is the target variable. The key question is whether the board structure and puzzle metadata provide enough information to estimate a puzzle's difficulty class without its rating. This is important for training tools that need to estimate difficulty for new puzzles that have not yet been rated.

I choose this dataset because it is sufficiently large for meaningful evaluation, contains a mix of numerical, categorical, and board-derived features, and the target variable has a natural ordinal structure. The four classes are not equally separable, and the class boundaries are defined on a continuous scale, as discussed in the evaluation chapter.

1.2 Objectives of the Project

The objectives of this project are the following:

- Constructing a full data mining pipeline, starting with a raw CSV file and ending with evaluated models, so that it can run reproducibly on Google Colab.
- It is possible to determine the difficulty class of a chess puzzle—classifying it as Easy, Medium, Hard, or Expert—on the basis of the board features and the puzzle metadata, without making use of the rating value itself.
- Applying descriptive mining in the shape of association rule analysis between tactical themes and difficulty classes can be done using the Apriori algorithm [2].

- We compared six classifiers—these included a baseline, two tree ensembles, a method based on neighbours, and two support vector machines—using the same evaluation procedure and taking the macro averaged F1 score as the main metric.
- Auditing the trained models for shortcut learning by eliminating any potentially leaking features and observing the resulting drop in performance.
- To examine the mistakes of the best model and to explain where the remaining accuracy is being lost.

1.3 Tools and Environment

The whole project is implemented in a single Jupyter notebook (`analysis.ipynb`) with 35 cells, executed on Google Colab with a standard CPU runtime. The main libraries are pandas and NumPy for data handling, scikit-learn for the models and the evaluation protocol, python-chess for reading chess positions, and mlxtend for the Apriori implementation. Charts are drawn with matplotlib. The dataset is stored in Google Drive and mounted at the start of the notebook, so every run uses the same 50,000 puzzle sample.

CHAPTER - 2

DATASET DESCRIPTION AND EXPLORATORY DATA ANALYSIS

2.1 Dataset Overview

The dataset consists of a sample containing 50,000 rows from the Lichess puzzle database and is stored in the file `lichess_db_puzzle_sample.csv`; each row in the dataset corresponds to one puzzle, and Table 2.1 provides the names of the eleven columns of the original file.

Table 2.1: Columns of the raw dataset

Column	Type	Description
PuzzleId	Text	Unique identifier of the puzzle
FEN	Text	Board position before the solution, in Forsyth-Edwards notation
Moves	Text	Solution move sequence in UCI notation
Rating	Integer	Glicko difficulty rating of the puzzle (target source)
RatingDeviation	Integer	Uncertainty of the rating; high values mean few plays
Popularity	Integer	User vote score between 0 and 100
NbPlays	Integer	Number of times the puzzle was played
Themes	Text	Space separated tactical theme tags
GameUrl	Text	Link to the original game
OpeningTags	Text	Opening names (mostly empty)
DailyDate	Text	Date field (almost entirely empty)

2.2 Data Quality Check

The initial data quality check assessed missing values and duplicate rows. The dataset contained no duplicates, but two columns were largely empty: OpeningTags was missing in 79.80% of rows and DailyDate in 99.97%. Both columns, along with identifier and link columns that have no predictive value, were dropped during preprocessing. The three numeric metadata columns (RatingDeviation, Popularity, NbPlays) and the two relevant text columns (FEN and Themes) are complete with no missing values.

2.3 Target Variable and Difficulty Classes

The ratings range from 399 to 3108, having a mean of approximately 1472 and a standard deviation of 546.5. In the case of the classification task, this continuous scale is divided into four difficulty categories using predetermined thresholds: Easy for ratings of 1200 or less, Medium for those up to 1600, Hard for ratings up to 2000, and Expert for ratings above 2000. The thresholds were set before any modelling was carried out, meaning that the definition of the classes is in no way dependent on the models.

In Figure 2.1 the rating histogram is shown with the three cut-off points indicated. The distribution has a long tail on the right with a small number of puzzles lying above 2800. Figure 2.2 and Table 2.2 display the class distribution obtained. The classes are imbalanced since the Easy class is the largest, making up 36.8% of the puzzles, then comes the Medium class at 23.9%, followed by the Hard class with 20.2% and finally the Expert class with 19.1%. It is because of this imbalance that the project employs macro averaged F1 as its main metric rather than plain accuracy, a point to which I shall return in Chapter 4.

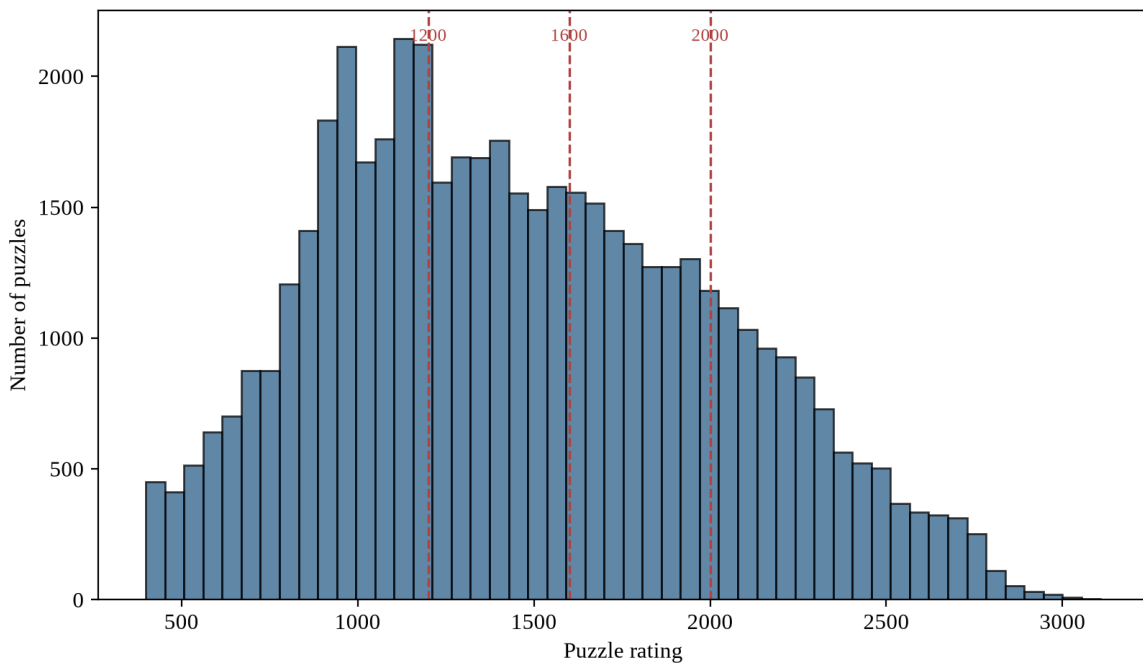


Figure 2.1: Rating distribution with the class boundaries at 1200 / 1600 / 2000

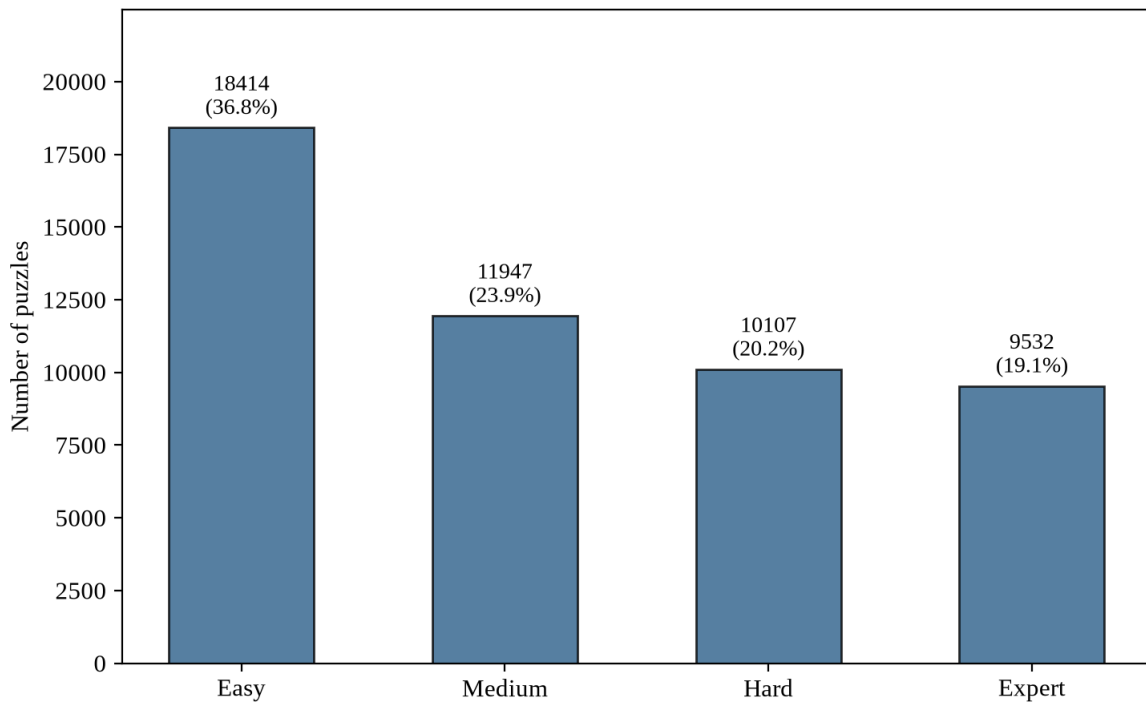


Figure 2.2: Distribution of difficulty classes

The rating is binned and the class counts are as shown in Table 2.2.

Table 2.2: Distribution of difficulty classes

Class	Rating range	Count	Share
Easy	≤ 1200	18,414	36.8%
Medium	1201 - 1600	11,947	23.9%
Hard	1601 - 2000	10,107	20.2%
Expert	> 2000	9,532	19.1%

2.4 Relationships Between Features

Figure 2.3 examines the behavior of the two popularity-related metadata columns across the rating scale. Popularity remains high across most ratings, dropping only at the extremes, while the number of plays increases with rating up to the mid-range before declining. The second panel uses a logarithmic scale due to the wide range of play counts, from a few to 175,098, with a median of 385. Both columns provide information about the rating, which is important to consider before modeling and raises the cold start issue discussed in the limitations chapter.

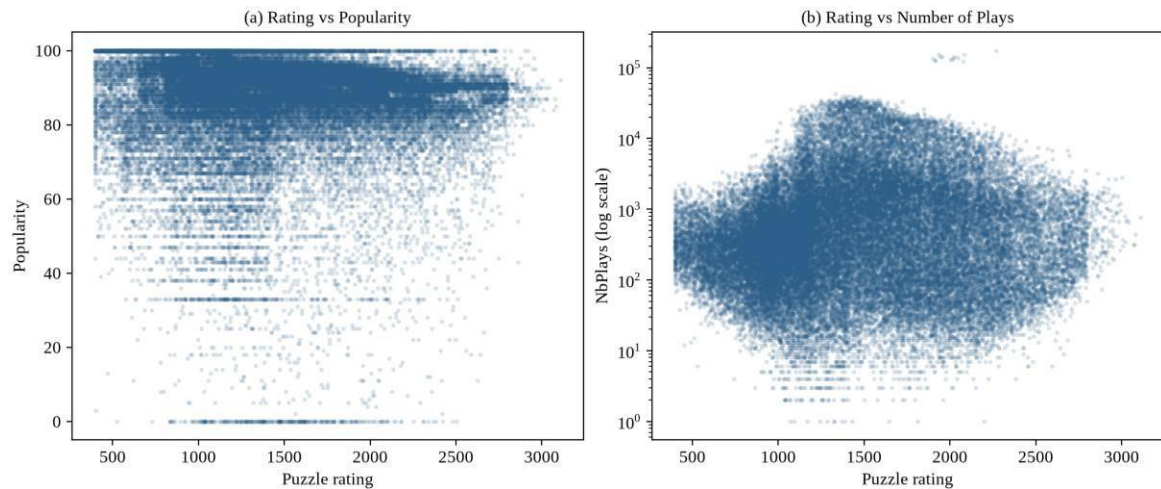


Figure 2.3: Rating against Popularity (a) and Number of Plays (b)

The Themes column includes 73 distinct tactical tags. Figure 2.4 displays the twenty most frequent tags. The most common tags are structural, such as mate, middlegame, short, and long, which describe the solution line, while others like forks, pins, and discovered attacks describe motifs. Since several tags directly encode the solution length, they require further examination before being used as features, as discussed in Section 2.5.

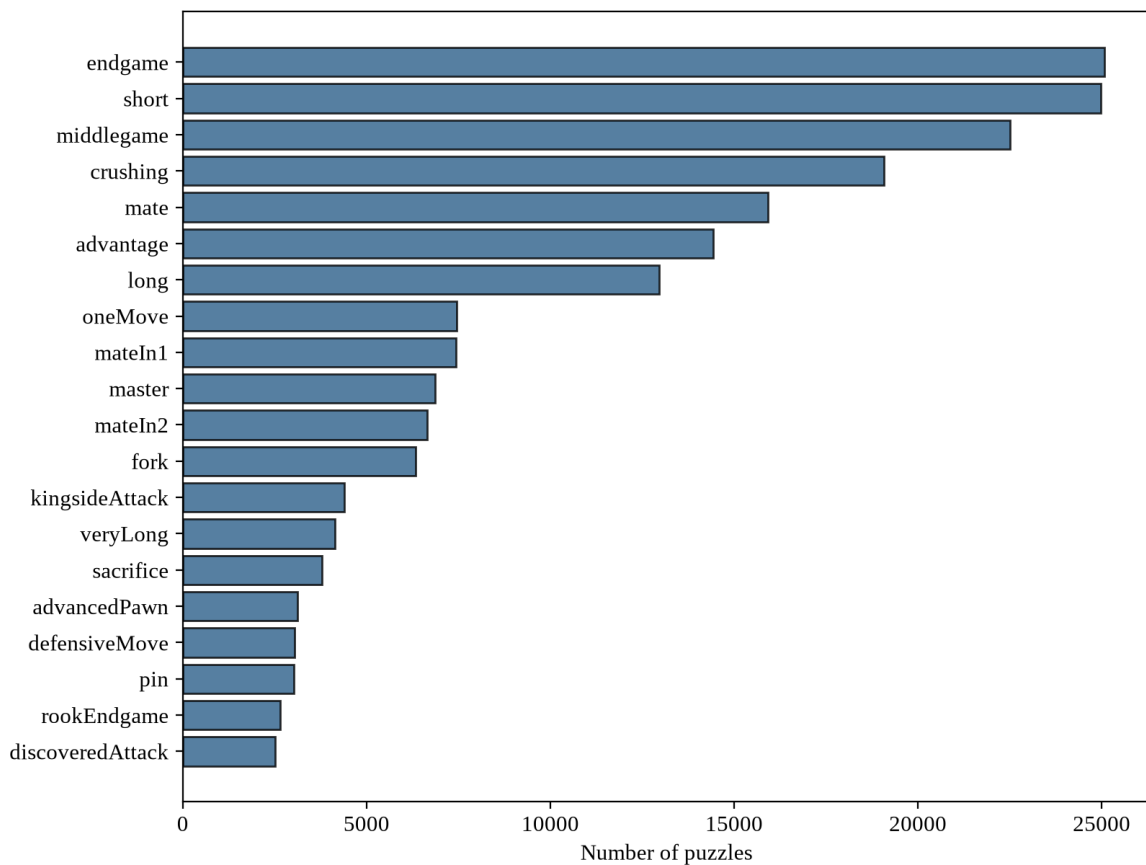


Figure 2.4: Twenty most frequent themes in the 50,000 puzzles

The correlation matrix for the four numeric columns is shown in Figure 2.5. There is a moderate correlation between Popularity and NbPlays, and RatingDeviation correlates negatively with both of these, which is reasonable since a puzzle that has been played thousands of times has a stable and low deviation rating whereas a new puzzle has a high deviation. The rating itself is not strongly correlated with any of the numeric columns, a fact which already suggests that the classification task will not be easy.

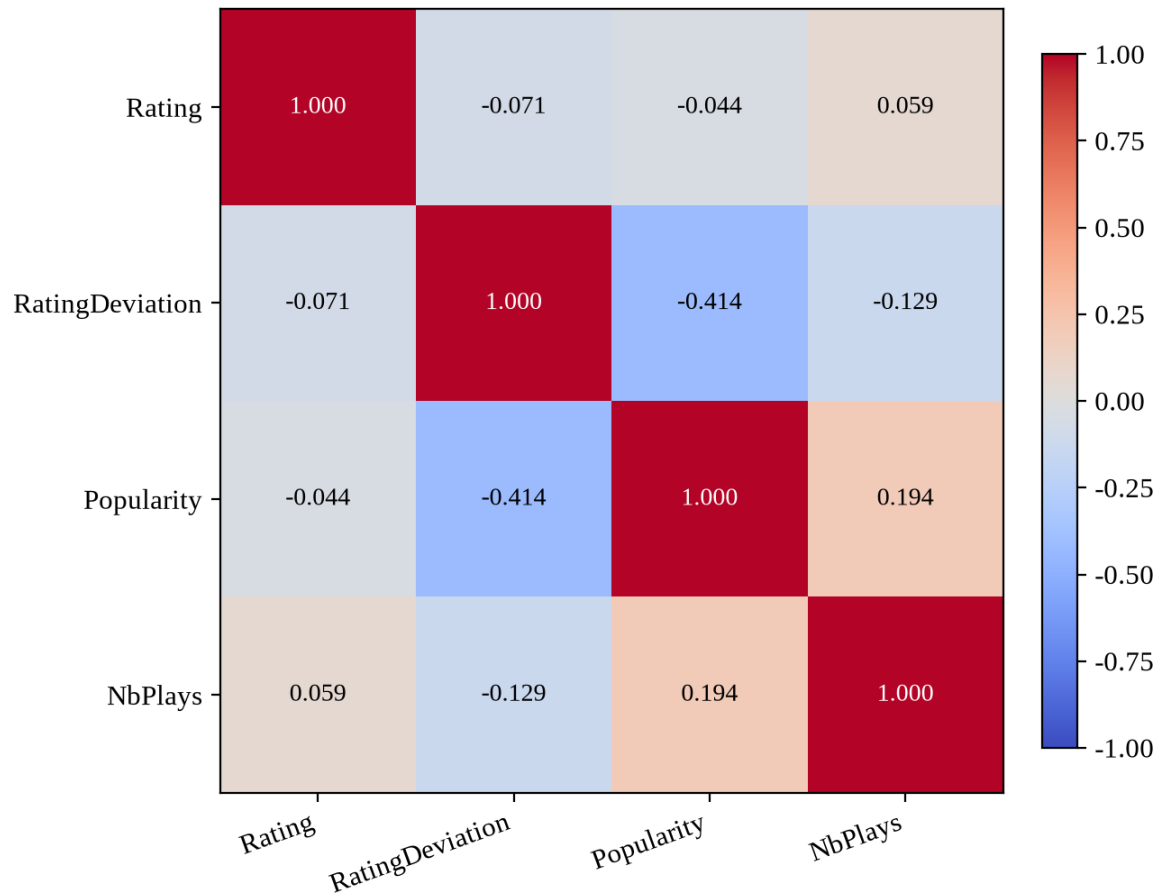


Figure 2.5: Correlation of the numeric columns

2.5 Potentially Leaking Themes

Eight tags are tautological because they reveal the answer length, making it easier to classify difficulty without chess knowledge. As the puzzle gets longer, the proportion of long and veryLong puzzles rises. A model that latches onto these flags would partly learn the answer length instead of the position. I did not remove them from the main feature set, because they are legitimately present in real puzzle metadata, but Chapter 4 contains a dedicated audit (Section 4.6) where all eight tags are removed and the performance drop is measured. This way the model quality with and without the leaking features is known exactly.

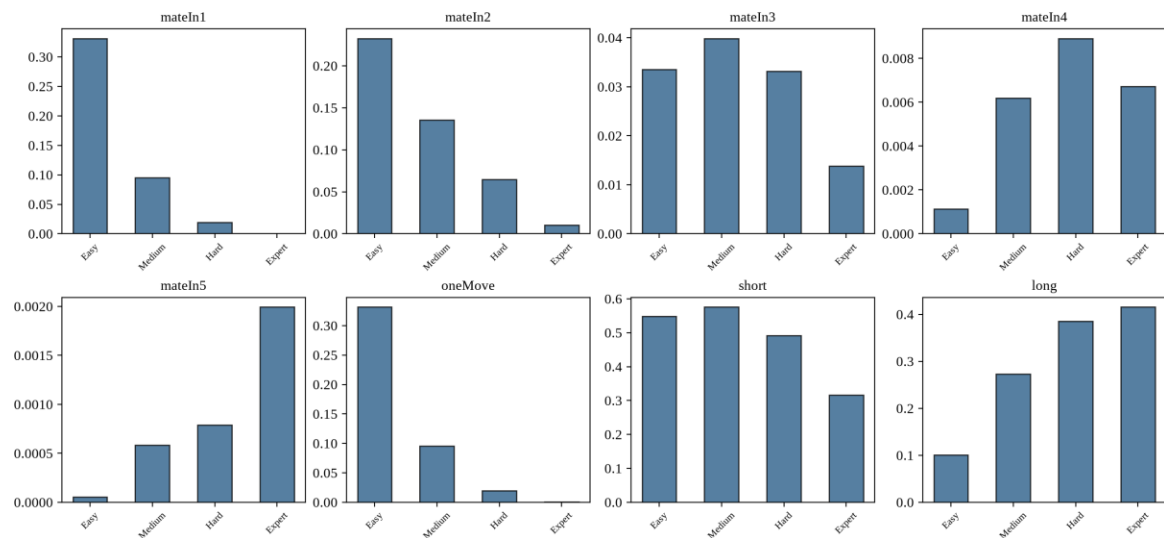


Figure 2.6: The proportion of each solution length theme within each difficulty class

CHAPTER - 3

DATA PREPROCESSING AND DESCRIPTIVE MINING

3.1 The Removal of Columns and the Process of Feature Engineering

Preprocessing begins by dropping six columns: PuzzleId, GameUrl, and Moves (identifiers or links), OpeningTags and DailyDate (mostly empty), and Rating (the target variable, which must not be included as a feature). This leaves three numeric metadata columns, the FEN string, and the Themes. The FEN string enables extraction of chess-specific features. Using the python-chess library, I developed a function to read each position and extract fourteen board features, as listed in Table 3.1. These features capture material balance, mobility, and king safety using structural measures rather than full engine analysis, which is beyond the project's scope. All 50,000 positions were parsed without errors.

Table 3.1: A list of the board features obtained from the FEN string

Feature	Description
WhiteMaterial / BlackMaterial	Sum of piece values (P=1, N=B=3, R=5, Q=9) per side
MaterialBalance	White material minus black material
TotalPieces	Number of pieces on the board
WhitePawns / BlackPawns	Pawn count per side
IsWhiteTurn	1 if White moves next, else 0
LegalMoveCount	Number of legal moves in the position (mobility)
InCheck	1 if the side to move is in check
WhiteKingAttackers BlackKingAttackers	/ How many enemy pieces attack each king
KingDistance	Chebyshev distance between the two kings
WhiteCastled	1 if the white king stands on the kingside files g or h
WhiteDoubledPawns	Number of files with two or more white pawns

3.2 Encoding, Scaling and the Final Feature Matrix

The Themes column is one-hot encoded, with each of the 73 theme tags represented as a binary column. Each puzzle receives a value of 1 for every tag present in its theme list. Combined with the three metadata columns and fourteen board features, the final feature matrix contains 90 features. After an 80/20 stratified split, the training set contains 40,000 rows and the test set 10,000 rows, with class proportions maintained and served.

The seventeen numerical features (that is, the three metadata columns together with the fourteen board features) are standardised using the `StandardScaler`, and the scaler is fitted only on the training set. The 73 theme columns are already binary so they are not altered at all. Although the fact that the scaler is fitted only on the training set may seem like a minor point, it is in fact important since if the scaler had been fitted on the entire dataset it would have leaked information from the test set into the training features.

3.3 The Use of the Apriori Algorithm for Association Rule Mining

The descriptive aspect of the project searches for combinations of themes that occur together and then determines which of these combinations indicate a particular difficulty class. In the usual form stated by Agrawal and Srikant, a rule is given a measure of support (that is, how frequently the combination appears), a measure of confidence (that is, how often the consequence is true when the combination is present), and a lift (that is, how much more frequently the consequence occurs in conjunction with the combination than it would by chance).

The matrix used as input includes all 50,000 puzzles' 73 theme columns together with four one-hot difficulty columns. Using a minimum support value of 0.05, the Apriori algorithm generates 153 frequent itemsets, comprising 24 single items, 70 pairs, 45 triples and 14 more complex combinations. Of these, only those rules which have a confidence of 0.70 or higher and a lift above 1.0 are selected, and from those only the rules that conclude with a difficulty class are kept; this results in 42 rules, all of which conclude with the Easy class, about which I will talk below.

In Figure 3.1 the 42 rules are shown with their support plotted on the horizontal axis, confidence on the vertical axis and lift indicated by colour. The rules group together in a band of moderate support ranging from 0.05 to 0.12 and high confidence ranging from 0.7 to 0.9.

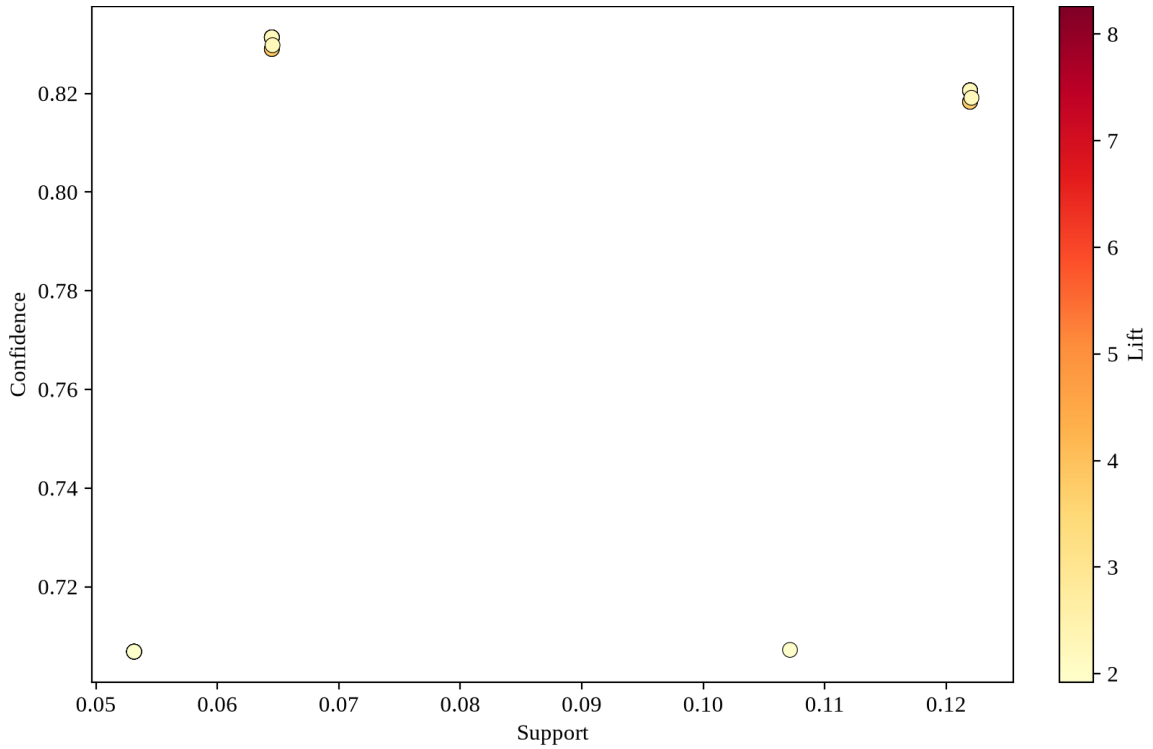


Figure 3.1: The 42 mined rules, with support against confidence and coloured by lift

3.4 Rules Indicating Difficulty Classes

The strongest rules by lift are given in Table 3.2. The pattern is the same in that those combinations which include short-solution markers (`mateIn1`, `mateIn2`, `oneMove`) together with phase tags (`endgame`, `middlegame`) indicate the Easy class and have lifts ranging from about 4 to 8.3. At this support level no rule with a confidence greater than 0.70 indicates that a puzzle is either Hard or Expert. Although Hard puzzles do have typical themes, no one particular frequent theme combination is characteristic of the Hard class. Difficulty exceeding 2000 depends on the position and on the precision required, and no single tag that is shared by most of the puzzles captures this.

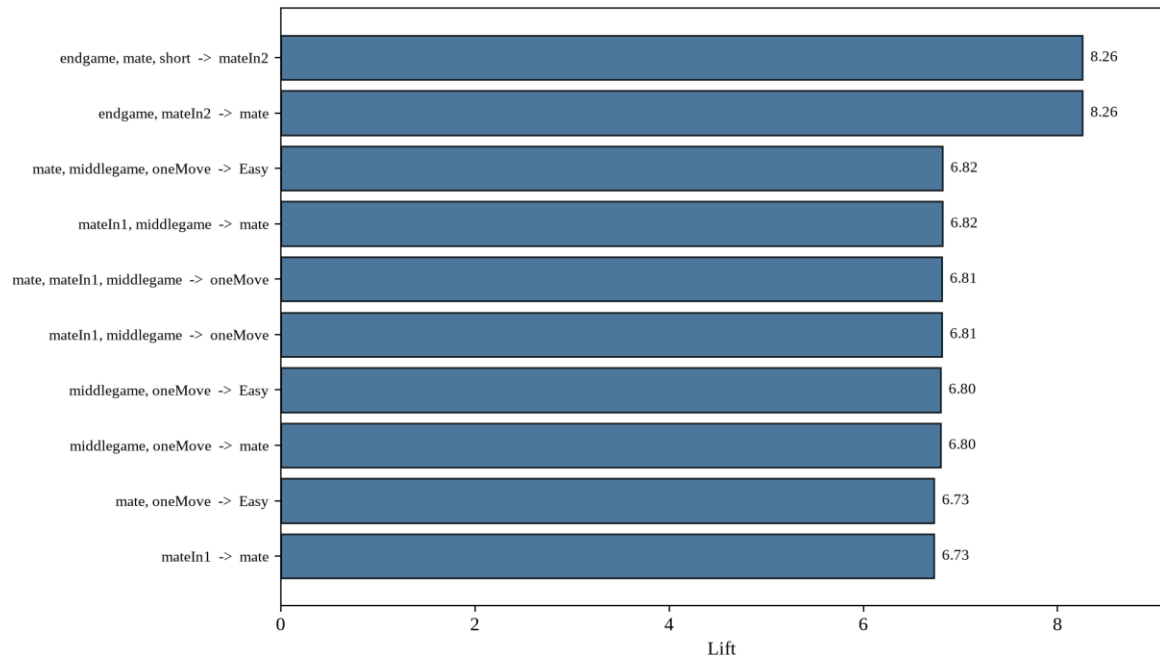


Figure 3.2: Ten strongest rules by lift

Table 3.2: Strongest association rules with a difficulty class in the consequent

Antecedents	Consequent	Support	Confidence	Lift
endgame, mateIn2	Easy	0.053	0.707	8.26
endgame, mate, short	Easy	0.053	0.707	8.26
mateIn1, middlegame	Easy	0.064	0.831	6.82
mate, mateIn1, middlegame	Easy	0.064	0.831	6.82
mate, middlegame, oneMove	Easy	0.064	0.831	6.82
middlegame, oneMove	Easy	0.080	0.805	6.60
mateIn2, middlegame	Easy	0.087	0.787	6.46
mate, mateIn2, middlegame	Easy	0.087	0.787	6.46
endgame, mateIn2, short	Easy	0.053	0.707	6.35
endgame, mateIn3	Easy	0.079	0.710	5.74

Two conclusions came out of this analysis. First, the descriptive and predictive views agree: the same solution-length tags that dominate the rule list are the ones flagged as tautological in Section 2.5, so the rules largely re-describe the answer length. Second, the absence of Hard and Expert rules means difficulty prediction needs models that combine many weak signals; simple rule based reasoning cannot capture it.

CHAPTER - 4

PREDICTIVE MINING AND EVALUATION

4.1 Models and Hyperparameter Search

Six classifiers are trained and compared under one protocol: a 5-fold stratified cross validation on the 40,000 training rows with macro F1 as the scoring metric. The Dummy classifier always predicts the most frequent class and exists only to set the lower bound. Decision Tree and Random Forest represent single trees and bagged ensembles, KNN is an instance based method, and LinearSVC and RBF SVM cover the linear and kernelised versions of support vector machines. The RBF SVM is tuned and evaluated on a stratified random subsample of 10,000 training rows with 3-fold cross validation, because a full grid on all 40,000 rows was not feasible on the available CPU. Table 4.1 lists the search grids.

Table 4.1: Hyperparameter grids used with GridSearchCV

Model	Parameters searched	Best found
Dummy	strategy = most_frequent	—
Decision Tree	max_depth in {10, 15, 20, None}; min_samples_split in {2, 5, 10}	max_depth = 10, min_samples_split = 5
Random Forest	n_estimators in {100, 200, 300}; max_depth in {10, 20, None}	n_estimators = 200, max_depth = 20
KNN	n_neighbors in {3, 5, 7, 9}; weights in {uniform, distance}	n_neighbors = 9, weights = distance
LinearSVC	C in {0.1, 1, 10, 100}; max_iter = 10000	C = 1
RBF SVM	C in {1, 10, 100}; gamma in {scale, auto}	C = 1, gamma = scale

4.2 A Problem That Was Resolved While Mining

The initial working version of the training cell failed during its very first execution. When running on the free Colab runtime the session crashed halfway through the first grid search and the log ended with thousands of repeated deprecation warnings from the notebook backend rather than showing the expected result lines. I initially thought the problem might be due to the dataset size, but the complete feature matrix consists of only about 40,000 rows by 90 columns, or roughly 36 MB in memory, an amount well below what a Colab session is capable of holding. In fact, there were three errors.

In the first place, my code included nested parallelism since GridSearchCV was configured with `n_jobs = 2` and the Random Forest within it also had `n_jobs = 2`; this caused

each of the two search workers to create two additional threads of its own, resulting in four heavy-computing threads competing for just two CPU cores, with a separate copy of the data residing in each worker process. Second, the LinearSVC with `max_iter = 10000` could take an unreasonably long time when processing the larger values of `C`. Third, one long cell that did not produce any output combined all six of the models, meaning that if a failure occurred, all the previous work would be lost.

I converted the training matrix to `float32`, which reduced the memory usage by half and increased the speed of all the models without affecting the results. I transferred all the parallelism to the grid search level (by setting `n_jobs` to `-1`) and set the estimators themselves to use `n_jobs = 1`, thus eliminating thread oversubscription. In the case of LinearSVC I specifically set `dual` to `False`, since this solves the primal problem and is the appropriate choice whenever the number of samples is much greater than the number of features. Lastly, I divided up the training into separate parts for each model, saved each completed model to a checkpoint folder on Google Drive using `joblib`, and skipped any model for which a checkpoint already existed when the cell was run again.

With the changes made to the training phase, all six models together with their full grids are completed in about six to ten minutes on a standard Colab runtime, and the only loss from a crashed session is incurred by the model which was running at the time of the crash. I have confirmed that the changes had no effect on the results: the cross validation scores from the old and the new training code agree to four decimal places for each model.

4.3 Cross-Validation Results

Table 4.2 gives the macro F1 score together with the standard deviation obtained from the five folds, and Figure 4.1 presents the same figures in the form of a bar chart. The Random Forest has the highest value at 0.5928, then comes the single Decision Tree at 0.5788. The difference between these two is small, which indicates that the tree structure itself contains most of the signal and that bagging provides only a minor improvement. KNN reaches 0.5263, the RBF SVM attains 0.5442, and the LinearSVC brings the real models to an end by scoring 0.4740. The Dummy baseline is at 0.1346.

Table 4.2: The macro F1 score (mean and standard deviation over 5 folds)

Model	CV macro F1 (mean)	Std
Dummy	0.1346	0.0000
Decision Tree	0.5788	0.0015
Random Forest	0.5928	0.0038
KNN	0.5263	0.0032

Model	CV macro F1 (mean)	Std
LinearSVC	0.4740	0.0044
RBF SVM (10k subsample)	0.5442	0.0070

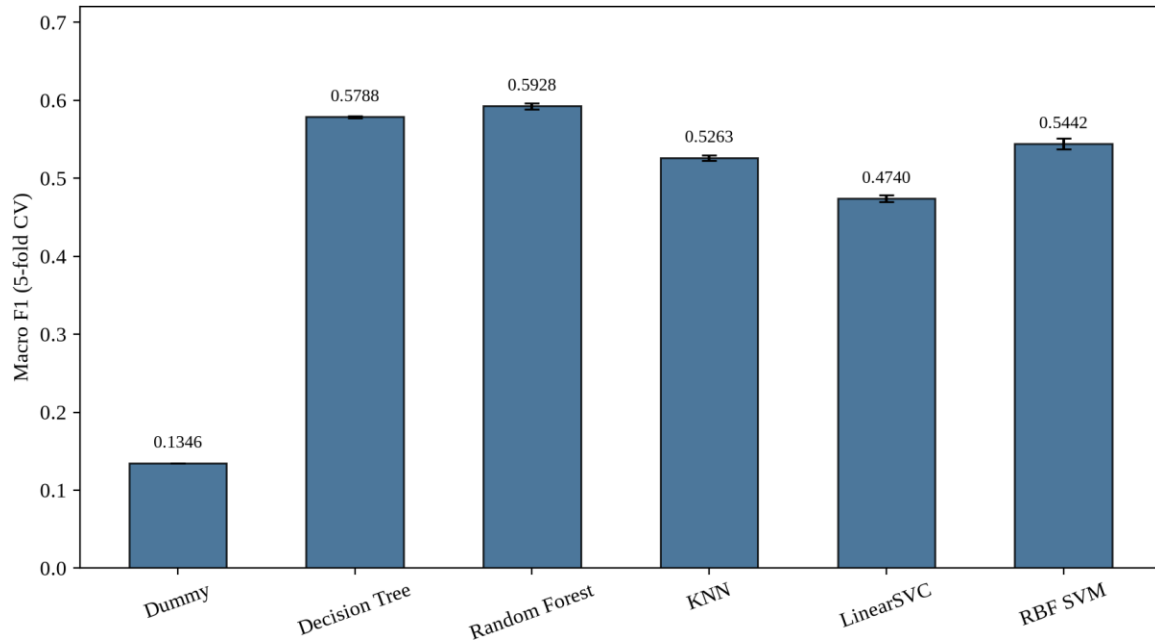


Figure 4.1: The macro F1 score with standard deviations calculated by cross-validation

It is the two different SVM versions that exhibit the greatest difference in this table; although they are based on the same data and use the same penalty grid, the kernel version beats the linear one by seven percentage points, since the class boundaries in the feature space are not linear and any model capable only of drawing hyperplanes has to suffer the consequences.

4.4 Test Set Results

Each model predicts the test set consisting of the 10,000 rows that were not used in the training phase exactly once. The four metrics for each model are given in Table 4.3, and the figures correspond to the order established by the cross-validation: the Random Forest is the best performing model, achieving an accuracy of 62.83% and a macro F1 score of 0.5909, this being almost the same as its cross-validation score of 0.5928, which shows that there is no gap between the performance when the model was tuned and when it was finally evaluated.

Table 4.3: The results from the test set of all six models (10,000 held-out puzzles)

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
Dummy	0.3683	0.0921	0.2500	0.1346
Decision Tree	0.6088	0.5855	0.5754	0.5761
Random Forest	0.6283	0.5930	0.5920	0.5909
KNN	0.5671	0.5298	0.5321	0.5306
LinearSVC	0.5390	0.4842	0.4875	0.4700
RBF SVM	0.5859	0.5539	0.5476	0.5499

Regarding why macro F1 is the figure given: this is because Easy makes up 36.8% of the puzzles, so a classifier that always assigned instances to that class would achieve an accuracy of 36.8%, which is precisely what the Dummy row illustrates. Macro averaging gives equal weight to all four classes, and this is the more appropriate approach in this case. Compared with the Dummy macro F1 of 0.1346, the Random Forest improves the balanced score by a factor of about 4.4.

The confusion matrices for all six models are presented in Figure 4.2, arranged in order of increasing difficulty from Easy at the top to Expert at the bottom. A clear pattern is that almost all of the mass lies on or adjacent to the diagonal. Figure 4.3 displays the one-vs-rest ROC curves for the same models. The Random Forest separates the Easy class almost cleanly, achieving an AUC of 0.924, and even the most difficult class, Medium, remains close to 0.78. This adjacency effect is quantified in Section 4.7.

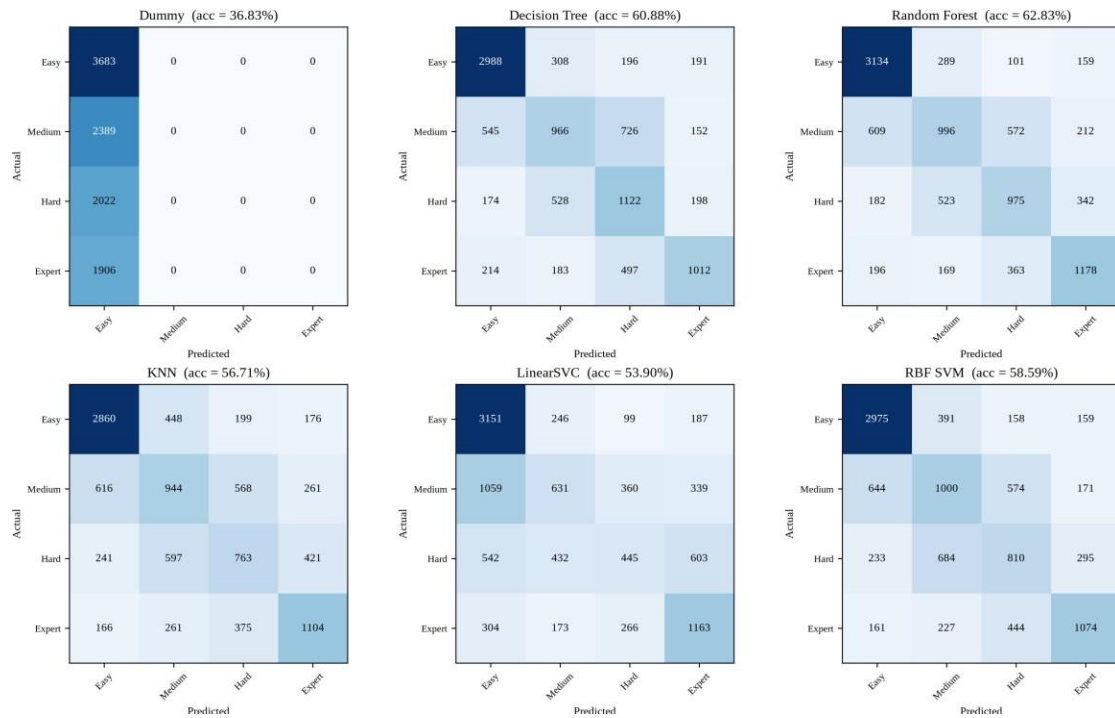


Figure 4.2: Confusion matrices of the six models on the test set

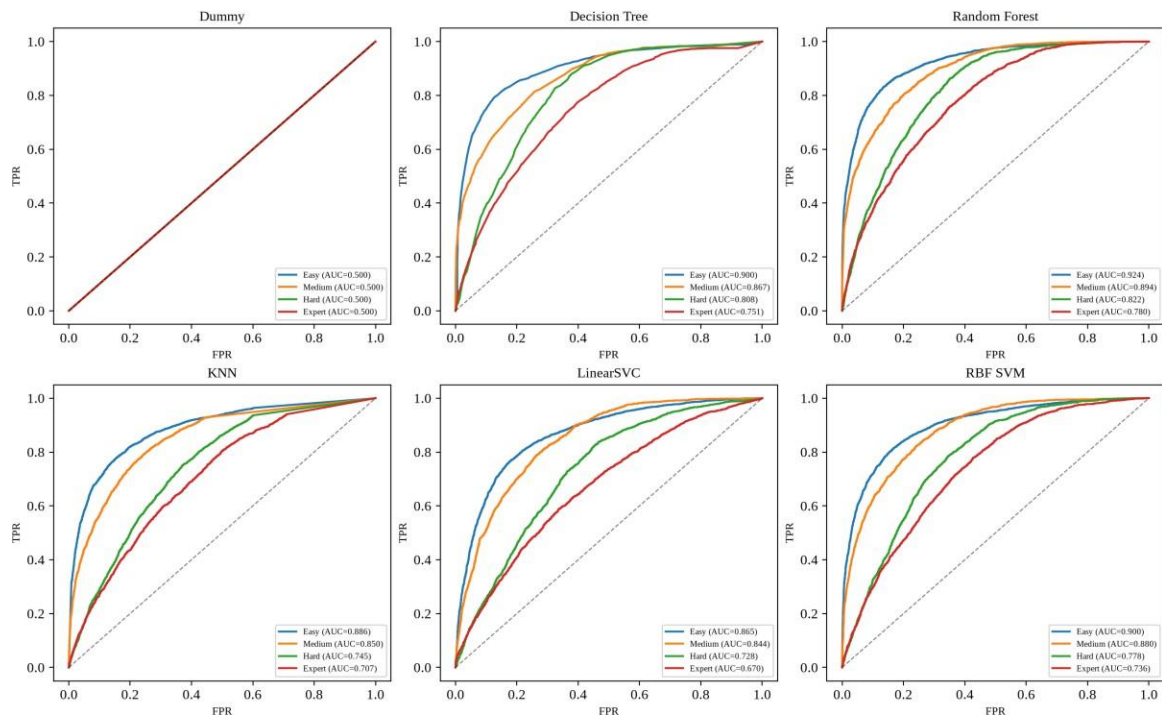


Figure 4.3: One-vs-rest ROC curves per model

4.5 Feature Importance

The fifteen most important features of the Random Forest are listed in Figure 4.4; the top three of these are the metadata columns—NbPlays, RatingDeviation and Popularity—then come the mate theme flag and the fork theme flag; the board features (material balance,

piece counts, mobility) are in the middle of the ranking and have individually very small but non-zero contributions.

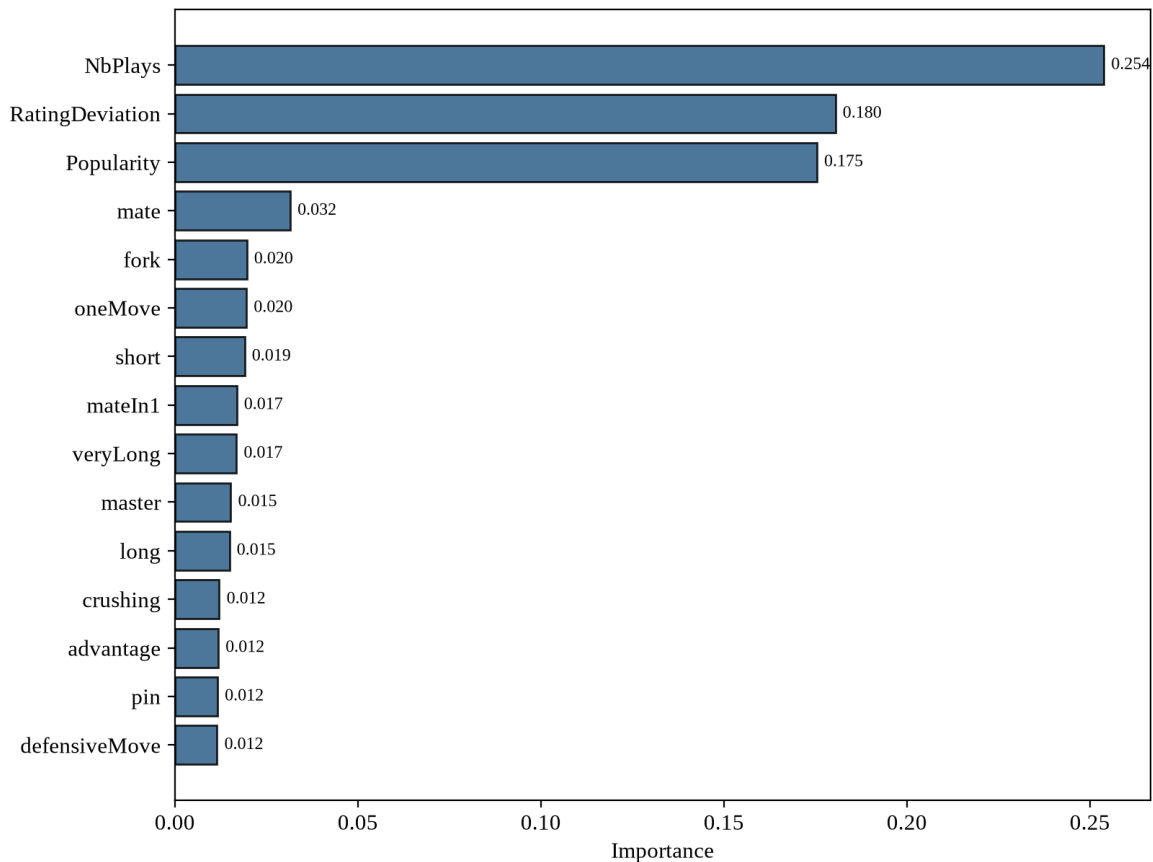


Figure 4.4: The fifteen most important features of the Random Forest

The metadata relating to popularity shows how the crowd has so far reacted to a puzzle. In the case of puzzles which have already been played thousands of times, this information is reasonable. However, for a puzzle that was published just yesterday, NbPlays and RatingDeviation appear completely different and as a result the model loses its three strongest signals. The leakage audit described in the following section looks at a similar risk in relation to the theme.

4.6 The Leakage Audit: Eliminating the Themes Related to the Solution Length

The eight theme tags mentioned in Section 2.5 are capable of encoding the length of the solution line directly. In order to assess how much the model relies on these tags, I retrained the same Random Forest model two times using the same data split: in Run A I used the complete set of features, and in Run B I excluded the eight tautological theme columns. The results of this are given in Table 4.4.

Table 4.4: Checking for leakage in the Random Forest using the same split and the same settings

Run	Feature set	Accuracy	Macro F1
Run A	All 90 features	0.6144	0.5805
Run B	82 features, 8 tautological themes removed	0.6031	0.5694
Drop	—	1.13 pp	1.10 pp

The decrease of 1.13 percentage points is small. Although the model makes use of the solution-length tags, most of its performance is retained even when none of the eight columns are included, indicating that the model has learnt genuine positional and metadata signals. Yet the audit finds that the tautological features are not severe enough to warrant their removal from the main pipeline, and so both versions are left documented and reproducible.

4.7 Error Analysis

Out of the 10,000 test puzzles, the Random Forest incorrectly classifies 3,717 of them, and it is the nature of these errors that is more important than the number. If the difficulty classes had been purely arbitrary labels, the errors would be evenly distributed among all pairs of classes; as it is, they are not, since 72.6% of the errors occur between neighbouring classes, and when a prediction is permitted to be off by one bin, the model is correct in 89.8% of the cases. Mistakes that are distant—for instance, classifying an Easy puzzle as Expert—are rare.

The per-class accuracy shows where the losses are concentrated: easy puzzles are recognised well (85.1%), expert puzzles are recognised reasonably (61.8%), and the two middle classes are around 42% (Medium) and 48% (Hard). This is the kind of behaviour that is to be expected from a binned continuous scale since puzzles rated 1580 and 1620 are essentially identical but end up in different classes because of the 1600 boundary, and no feature set that excludes the rating itself is able to reliably resolve that difference.

Table 4.5: Puzzles that were incorrectly classified (Random Forest, test set)

PuzzleId	Rating	True class	Predicted	Themes
czIDl	1490	Medium	Hard	advantage endgame short
CUVwr	2063	Expert	Hard	advantage deflection discoveredAttack
VV6zH	2197	Expert	Hard	crushing middlegame short
Zvfgn	1304	Medium	Hard	crushing endgame long master
A2uyB	1430	Medium	Hard	middlegame

All five of the mistakes listed in Table 4.5 adhere to the same pattern: the ratings are near a class boundary and the predictions differ by exactly one bin. The puzzle rated 1490 which the model classifies as Hard is 110 points below the boundary, while the one rated 2197 and referred to as Hard is 197 points above it. Since no element of the feature set is able to distinguish these positions from their neighbours on the other side of the boundary, the remaining error is mostly irreducible with respect to this feature set.

4.8 The Implemented System

The notebook is the deliverable and is organised into six sequential stages which correspond to the sections of this report: data loading, exploratory analysis, preprocessing, association rule mining, model training and evaluation. It connects to Google Drive, reads the CSV from a predetermined path and writes each trained model to a checkpoint folder located in Drive together with a JSON file containing the results of the cross validation. Since each model checks for its own checkpoint file before training begins, the pipeline can be restarted after an interruption by merely rerunning the training cell, and completed models are never recomputed.

On a standard Colab CPU runtime a full run from CSV to the final evaluation tables takes under half an hour including preprocessing, and the training phase alone takes roughly six to ten minutes with the optimisations described in Section 4.2. The notebook, the dataset and this report form the complete submission package.

CHAPTER - 5

CONCLUSION

The project started with a raw download of 50,000 puzzles from the Lichess database and developed a classification system which estimates puzzle difficulty based on the board position and the metadata, without having any view of the rating value that it is attempting to predict. On the held-out test set, the Random Forest achieved an accuracy of 62.83% and a macro F1 score of 0.5909, this being 4.4 times better than the majority-class baseline when using the balanced metric, and the cross validation figures closely predicted the test result.

When looking at the descriptive mining from the other side, the same pattern appears: all the strongest association rules lead to the Easy class via short-solution themes, and there is no frequent rule combination that is characteristic of Hard or Expert puzzles. Difficulty exceeding the middle point of the scale isn't due to a single tag; rather, it results from the position and from metadata signals which only a model that combines many features is able to detect.

The analysis of the errors accounts for the figure in the headline. About three quarters of the mistakes fall into the adjacent class, and with a tolerance range of one bin the model gets the answers correct for 89.8 per cent of the test set. Since the classes are arbitrary divisions on a continuous rating scale and 17 per cent of all puzzles are within 50 rating points of a boundary, the rest of the errors is mainly a result of the way the task has been defined.

5.1 The Benefits of this Project

- The pipeline is fully reproducible since it uses only one notebook, a single fixed dataset, fixed random seeds, and all the numbers that have been reported can be regenerated from scratch.
- The document includes coverage of both types of mining: descriptive rule mining using Apriori and predictive modelling employing six classifiers all evaluated by the same evaluation protocol.
- The assessment makes use of macro F1 as its main metric since this is suitable for the imbalanced four-class target and includes the Dummy baseline in order that the lift can be seen.
- By carrying out a thorough leakage audit, genuine positional learning can be distinguished from shortcut features, and an error analysis based on tolerance reveals how the errors are distributed among the class boundaries.

5.2 The Disadvantages of This Project / Limitations

- Difficulty is a continuous variable that is grouped into bins. The fixed thresholds result in boundary problems which cannot be dealt with correctly without reference to the rating itself, and as a result the accuracy achievable is limited at the level that the best model attained.
- The top three factors are popularity metadata; in the case of new puzzles which have few plays these signals are weak or missing, so the system would have to undergo retraining or have its features adjusted before a genuine cold start deployment could take place.
- The board features are of a structural nature only; the pipeline does not have engine evaluation, tactical depth measures, or solution-line analysis, and this means that the model cannot follow exactly what makes a position difficult.
- The values for the class thresholds (1200 / 1600 / 2000) were determined by design. It is possible that a regression approach which predicts the rating first and then groups the results might yield better performance. It would also be worthwhile to try engine-derived features and models that analyse the move sequence.

REFERENCES

- [1] Lichess, "Lichess Open Database: Chess Puzzles," <https://database.lichess.org/#puzzles> (accessed August 2026).
- [2] N. Fiekas, "python-chess: A pure Python chess library," <https://python-chess.readthedocs.io> (accessed August 2026).
- [3] Google, "Google Colaboratory," <https://colab.research.google.com> (accessed August 2026).